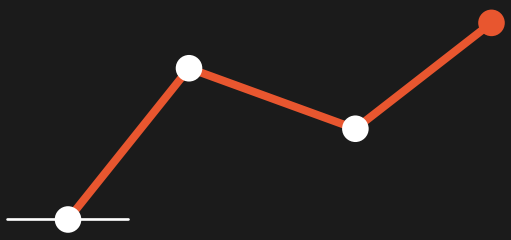
D1T-RESEARCH



PROJECT BOOK

THE D1 ARM, FROM SCRATCH

Characterizing and controlling a Unitree D1 robot arm — the code, the experiments, and the robotics ideas behind them

RAMIRO GONZALEZ

The D1 Arm, From Scratch

Characterizing and controlling a Unitree D1 robot arm — the code, the experiments, and the robotics ideas behind them

Ramiro Gonzalez

The D1 Arm, From Scratch

Working documentation for the **`D1T-Research`** repository, intended to be rebuilt at each phase freeze. This edition describes the system as of July 2026 (end of Phase 2, Phase 3 tooling complete) and was built from repository revision **`91c65d8-dirty`**. Code shown here is excerpted from the repository; when the book and the repository disagree, **the repository wins** on detail — the book's job is the reasoning.

Hardware: Unitree D1 arm (6 DOF + gripper)
Workstation: Ubuntu 26.04 LTS (Resolute Raccoon)
ROS 2 (Phase 7 target): Lyrical Luth
Theory text: *Modern Robotics*, Lynch & Park
Repository: **`D1T-Research`**

First assembled: July 2026

Table of Contents

Contents

Preface **7**

1 The Project and the Plan **9**

The goal . . . 9
The eleven phases . . . 9
The rules of work . . . 10
The hardware: D1-550 in numbers . . . 10
A tour of the repository . . . 10
Summary . . . 11

2 From Box to First Motion **13**

Technical requirements . . . 13
Step 1 — physical setup . . . 13
Step 2 — install unitree_sdk2 . . . 13
Step 3 — clone and run setup.sh . . . 14
Step 4 — install ROS 2 Lyrical Luth . . . 14
Step 5 — power on and verify, in order . . . 15
Step 6 — first commanded move . . . 15
About the D1-T dual-arm kit . . . 15
Troubleshooting . . . 16
Summary . . . 16

3 Talking to the Arm **17**

The physical link . . . 17
DDS in one page . . . 17
The JSON protocol . . . 18
Recap — the stack . . . 19
Summary . . . 19

4 The C++/Python Bridge **21**

The architecture decision . . . 21
joint_commander.cpp — the command side . . . 21
get_arm_joint_angle.cpp — the telemetry side . . . 22
Driving the bridge from Python . . . 23
Summary . . . 23

5 The Experiment Toolkit **25**

log_joints.py — the telemetry logger . . . 25
poses.py and send_pose.py — the command side . . . 26
limits.py — the contract . . . 27
Recap — the toolkit's shape . . . 27
Summary . . . 27

6 Phase 2 — What the Joints Actually Do **29**
Why characterize at all . . . 29
The step-response method . . . 29
The results . . . 30
Recap — what Phase 2 bought . . . 31
Summary . . . 31

7 Phase 3 — The Safe Envelope **33**
The question Phase 3 answers . . . 33
The pose set and its coverage . . . 33
run_sequence.py — the experiment driver . . . 34
analyze_sequences.py — the verdict . . . 34
Exit criteria . . . 35
Summary . . . 35

8 The Road Ahead **37**
What to be comfortable with first . . . 37
Phase 4 — forward kinematics . . . 37
Phase 5 — Jacobians and differential IK . . . 37
Phase 6 — numerical IK . . . 38
Phase 7 — ROS 2 . . . 38
Phases 8–11 — trajectories, simulation, perception, capstone . . . 38
Summary . . . 39

A Quick Reference **41**
Environment . . . 41
Bring-up and smoke test . . . 41
Everyday commands . . . 41
Phase experiment workflows . . . 41
File map . . . 42
Protocol crib sheet . . . 42
Measured constants (Phase 2) . . . 42
Factory constants (D1-550 official spec) . . . 42
References . . . 43

Preface

This book documents a year-long project: taking a Unitree D1 robot arm from an unopened box to a full robotics stack — forward kinematics, inverse kinematics, ROS 2, trajectories, simulation, and perception — with every layer built and verified by hand on real hardware.

It exists for two reasons. First, as a record: six months from now it will not be obvious why `j2` gets an extra -0.6° on negative targets, or why the logger starts two seconds before every command. The answers are in the data, and this book explains them. Second, as a teaching document: everything here was learned while doing it, and the book is written so that someone starting from general programming knowledge — but no robotics — can understand not just *what* the code does but *why* it is shaped the way it is.

The guiding rule of the whole project is borrowed from experimental science: **verify before optimize — if it's not measured, it's not real.** Commanded angles are not measured angles. Datasheets are not ground truth. Every phase of the project produces raw logs, a script that analyzes them, and a written record of what was found.

Who this book is for

The future maintainer of this repository (most likely its author), and any reader who wants to see what careful, from-first-principles robotics work looks like on hobbyist-priced hardware: real telemetry, real safety constraints, and real firmware quirks discovered by measurement.

You should be comfortable with Python and a terminal. C++ appears only in one short bridge program, explained line by line. The robotics math (screw theory, Jacobians) is introduced conceptually in Chapter 8 before the phases that need it.

A note on change

The repository is alive: file names, command-line flags, pose tables, and thresholds will drift as the phases advance, and this book will periodically be behind. That is expected and managed rather than fought:

- **Concepts over coordinates.** Chapters lead with ideas that do not expire — step response, deadband, path dependence, screw axes, pub/sub — and use today's code as the worked example. If a file is renamed, read the excerpt for its *role* ("the pose sender", "the limits contract") and find the current file playing that role.
- **Volatile detail lives in one place.** Exact commands, filenames, versions, and measured constants are concentrated in Appendix A, so keeping the book current means revising one appendix, not seven chapters.
- **Editions track phase freezes.** The book is rebuilt at each phase tag, and the colophon records the exact repository revision an edition describes. For the true state of any detail, check out that revision — the repository always wins.

What this book covers

Chapter 1, *The Project and the Plan*, sets out the goal, the eleven-phase roadmap, the rules of work, and a tour of the repository.

Chapter 2, ***From Box to First Motion***, is the hands-on setup guide: hardware, Ubuntu 26.04, the SDK build, ROS 2 Lyrical Luth, the power-on ritual, and a verified first move — with troubleshooting.
Chapter 3, ***Talking to the Arm***, explains the communication stack: Ethernet, DDS publish/subscribe, and the JSON command protocol the arm actually speaks.
Chapter 4, ***The C++/Python Bridge***, walks through the two small C++ programs that connect Python to the robot, and the architecture decision behind them.
Chapter 5, ***The Experiment Toolkit***, covers the Python tools every experiment uses: the telemetry logger, the pose sender, and the safety-limits module.
Chapter 6, ***Phase 2 — What the Joints Actually Do***, presents the single-joint characterization campaign: step responses, settle times, zero offsets, and the discovery of a controller deadband.
Chapter 7, ***Phase 3 — The Safe Envelope***, covers multi-joint testing: named poses, sequence runs, path-dependence analysis, and the **`limits.py`** contract that all later phases import.
Chapter 8, ***The Road Ahead***, previews the robotics theory of Phases 4–11 — kinematics, Jacobians, IK, ROS 2, trajectories, simulation, perception — and what to read for each.
Appendix A, ***Quick Reference***, is the cheat sheet: commands, file map, and protocol tables.

1

The Project and the Plan

A robot arm on a desk is a promise, not a capability. Out of the box, the Unitree D1 will move to angles you send it — but it will not tell you how accurately, how repeatably, or how safely. The gap between "it moves" and "I can build a manipulation system on it" is precisely the work this project does, and this chapter explains how that work is organized.

The goal

The end state, targeted for mid-2027, is a complete perception-guided pick-and-place system: a camera detects an object, inverse kinematics — implemented from the math, not from a library — plans a reach, a trajectory generator executes it smoothly, and ROS 2 ties it all together. Every layer is validated on real hardware and published as a reproducible portfolio.

That target was chosen deliberately. Surveying what robotics employers ask for produces a consistent list, and the project phases map onto it directly:

- **ROS 2** (pub/sub, actions, tf2) — in roughly 90% of job postings; covered by Phase 7.
- **FK / IK from scratch** — demonstrates mathematical depth; Phases 4–6.
- **Trajectory planning** — Phase 8.
- **Simulation** (MuJoCo / Gazebo) — sim-to-real workflows; Phase 9.
- **Perception** — the top skill gap in manipulation; Phase 10.
- **Real hardware demos** — what separates hardware people from pure-simulation people; every phase here runs on the real arm.

The eleven phases

The roadmap lives in **`yearplan.md`** and is the single source of truth for scope. One phase is active at a time; a closed phase only reopens with evidence that something it produced is wrong.

1. **Phase 0–1** — OS setup, network bring-up, logging, zero and reference integrity. *Done.*
2. **Phase 2** — single-joint characterization: what does each joint actually do when commanded? *Done; results in Chapter 6.*
3. **Phase 3** — multi-joint coordination and the safe envelope. *Tooling complete; hardware runs in progress (Chapter 7).*
4. **Phases 4–6** — forward kinematics, Jacobians, numerical IK, all implemented from *Modern Robotics* (Lynch & Park) and validated physically.
5. **Phase 7** — ROS 2 integration; the stdin/stdout bridge is replaced by proper nodes and topics.
6. **Phases 8–10** — trajectories and gravity compensation, simulation, camera perception.
7. **Phase 11** — the capstone pick-and-place, measured over 20 trials.

The rules of work

Five rules govern every phase. They sound obvious; the discipline is in actually following them.

- **Verify before optimize.** If it's not measured, it's not real.
- **One variable per experiment.** Phase 2 moves one joint at a time for exactly this reason.
- **Freeze means freeze.** Each phase ends with a tagged commit and archived logs. Raw logs are immutable.
- **Every phase produces a public artifact** — code, a plot, and a short write-up.
- **Safety first.** If the arm strains, sounds different, or hits anything, that angle is out of bounds and gets documented. Never test near joint limits.

The hardware: D1-550 in numbers

The arm is a Unitree **D1-550** (the D1-T kit pairs two of them — more on that in Chapter 2). The numbers that shape this project, from the official specification:

- **Geometry**: 550 mm arm length (670 mm with gripper), 550 mm working radius, 3.15 kg total weight.
- **Payload**: 500 g rated load *including* the gripper jaws — a light-duty arm; gravity effects matter (Phase 8).
- **Joint torque**: 3.3 Nm on j0 and j1, 1.7 Nm on the rest. j1 carries the arm's own weight at reach, which is why it is the prime candidate for gravity compensation.
- **Factory range of motion**: $\pm 135^\circ$ on j0, j3, j5; $\pm 90^\circ$ on j1, j2, j4; 0–65 mm claw stroke on j6. This project's conservative envelope ($\pm 45^\circ$, Chapter 7) sits far inside these — deliberately. Never test near the factory limits.
- **Control**: onboard controller (4× Cortex-A55), DDS subscription over 100 Mbps Ethernet, **10 Hz control cycle** — the same 10 Hz seen in the telemetry stream and the streaming command mode.
- **Power**: 24 V / 10 A supply (240 W).

A tour of the repository

```
D1T-Research/
  setup.sh           one-command machine setup: network, NIC patch, build
  yearplan.md        the eleven-phase roadmap (source of truth for scope)
  d1_guide.ipynb     narrative guide: hardware -> setup -> commands
  d1_sdk/
    src/             C++ programs that speak DDS to the arm
    build/           compiled binaries
    PROTOCOL.md      the arm's JSON command protocol, documented
  experiments/       Python: loggers, pose tools, phase scripts, results
  logs/              raw telemetry CSVs, one folder per phase (immutable)
  90b2525.../        URDF + meshes, needed from Phase 4 onward
  ROS2/              ROS 2 study notes for Phase 7
  book/              this book
```

The split matters: **`d1_sdk/`** is the only place C++ lives, **`experiments/`** is where all science

happens, and **`logs/`** is write-once. Analysis scripts read logs; they never edit them.

> **Note**
>
> The D1 is 6 DOF plus a gripper, and the wire protocol exposes **seven** angle channels. The official numbering runs from the base: **`j0`** (base yaw) through **`j5`** are the six arm joints, and **`j6`** drives the gripper (a 0–65 mm claw stroke behind an angle channel). This project's tooling treats all seven uniformly; the kinematics phases operate on **`j0`**–**`j5`**.

Summary

The project turns a bare D1 arm into a validated robotics stack over eleven phases, each producing measured evidence before the next begins. The repository separates C++ robot I/O, Python experiments, and immutable logs. With the map in hand, the next chapter gets you from an unopened box to a verified first motion.

2

From Box to First Motion

This chapter is the complete setup path: hardware on the desk, a fresh Ubuntu 26.04 install, through to the arm verifiably obeying a command. It assumes nothing beyond a working OS and follows the exact order that avoids the common traps. If a step fails, jump to the troubleshooting section at the end before improvising.

Technical requirements

- **Unitree D1-550 arm** with its 24 V power supply (the D1-T kit contains two arms; one is enough for everything before the teleoperation experiments — see the D1-T section below).
- **An Ethernet path** between workstation and arm: a built-in port or a USB-C/USB-A Ethernet adapter — both work, and **`setup.sh`** auto-detects either.
- **Workstation with Ubuntu 26.04 LTS** (Resolute Raccoon), with sudo rights and internet access for the installs.
- **A clear workspace**: the arm bolted or clamped to a surface, with a full arm's reach of empty space around it — no monitors, mugs, or cables inside the sweep.

> **Important**
>
> Set up the physical safety habit before the software: know where the power switch is, and keep a hand's reach to it during every first-time motion. An arm doing something unexpected is stopped with power, not with Ctrl-C — the arm's controller, not your process, is executing the move.

Step 1 — physical setup

1. Mount the arm on a stable surface. It must not be able to tip or walk when moving at full extension.
2. Connect the Ethernet cable from the arm to the workstation (or to the USB adapter).
3. Connect power but leave the arm **off** for now — the software side comes first, so that telemetry is watchable the moment the arm boots.

Step 2 — install unitree_sdk2

The arm's SDK examples build against **`unitree_sdk2`**, which also provides CycloneDDS (the middleware explained in Chapter 3). This is a one-time system-wide install:

```
$ sudo apt update
$ sudo apt install -y build-essential cmake git
$ git clone https://github.com/unitreerobotics/unitree_sdk2
```

```
$ cd unitree_sdk2 && mkdir build && cd build
$ cmake .. && sudo make install
```

Step 3 — clone and run setup.sh

```
$ git clone <this-repo-url> D1T-Research
$ cd D1T-Research
$ ./setup.sh
```

The script is idempotent — safe to re-run at any time — and does four jobs, printing a check mark per step:

1. Installs any missing system packages (compiler, cmake, NetworkManager tools).
2. Auto-detects the Ethernet interface facing the arm and writes a static IP profile named **`d1-robot`**: host **`192.168.123.10`**, robot at **`192.168.123.100`**.
3. Patches the detected interface name into every SDK source file (the DDS layer binds to the interface by name — see Chapter 3 for why).
4. Builds all binaries into **`d1_sdk/build/`**.

Also install the Python analysis stack used by every experiment:

```
$ pip install numpy matplotlib pandas
```

> **Tip**
> Swapped to a different USB Ethernet adapter? The interface name changed with it. Just re-run **`./setup.sh`**; re-detection and re-patching is the supported workflow.

Step 4 — install ROS 2 Lyrical Luth

ROS 2 is not needed until Phase 7, but installing it during setup is cheap and makes the machine future-complete. Ubuntu 26.04 pairs with the **Lyrical Luth** distribution; use the official binary packages (never build ROS from source for this project), following the current instructions at **`docs.ros.org`** for Lyrical Luth. The shape of the install:

```
$ sudo apt install ros-lyrical-desktop
$ sudo apt install python3-colcon-common-extensions
$ echo "source /opt/ros/lyrical/setup.bash" >> ~/.bashrc
```

Verify with **`ros2 topic list`** in a fresh terminal — it should run without error (and show only default topics for now). Nothing else in this book touches ROS 2 until Phase 7.

Step 5 — power on and verify, in order

Switch the arm on and **wait 60–90 seconds** — the controller boots slower than feels natural, and commands sent early vanish silently. Then verify in this exact order, from **`d1_sdk/build/`**:

1. **Telemetry first** — prove you can *hear* the arm before asking it to do anything:

   ```
   $ ./get_arm_joint_angle
   servo0_data:0.1, servo1_data:0.4, ... servo6_data:-0.2
   ```

   A stream of lines like this at roughly 10 per second means the network, DDS, and arm are all alive. No lines? Troubleshooting section, entry 1.
2. **Enable the joints** (locks the motors so they hold position): **`./joint_enable_control`**
3. **Zero the arm**: **`./arm_zero_control`** — the arm moves smoothly to its zero posture. Watch it; this is the first motion.

Step 6 — first commanded move

Do the first real move through the Python tooling, not raw binaries, because it validates angles against the safe limits before sending (Chapter 5 explains the machinery):

```
$ python experiments/send_pose.py reach_1
Sent: j0=0  j1=-5  j2=10  j3=0  j4=5  j5=0  j6=0
$ python experiments/send_pose.py zero
```

The arm should ease into a slight reach and back. If it did, the whole stack works end to end, and everything else in this book is available to you. New machines end here; a new *arm* should also get the Phase 2 characterization re-run (Chapter 6) — offsets and deadbands are per-unit facts, not model facts.

About the D1-T dual-arm kit

D1-T is Unitree's teleoperation bundle: *two* D1-550 arms — an *acquisition* arm fitted with a hand-held claw (you move it, it streams its joint angles) and an *execution* arm that mirrors them — networked through a router. Useful facts from the official docs, even for single-arm work:

- **Every arm ships at `192.168.123.100`**. To run two on one network, SSH into one (**`ssh ubuntu@192.168.123.100`**, password **`123`**) and change its IP in **`/etc/network`** — the convention is **`.99`** for the second arm.
- **The arm is itself a Linux box** running Unitree's **`marm_*`** services (**`controller`**, **`control`**, **`communication`**, **`subscripber`**); they can be stopped, disabled, or — for multi-arm setups on one router — rebuilt with per-arm topic names by editing **`marm_code/src`** on the arm.
- **Two ways to address two arms**: separate NICs on the workstation with one arm each (bind by interface name, exactly the **`Init(0, "...")`** pattern this repository already uses), or unique DDS topics per arm when everything shares one router.

This project runs a single arm in the execution-arm configuration, with its factory services untouched. The teleoperation and dual-arm workflows become relevant only if a data-collection phase (drag teaching, demonstrations) is added later.

Troubleshooting

1. **No telemetry lines.** In order: is the arm's boot wait over? Link up (**`ip link`** shows the interface **`UP`**)? Can you **`ping 192.168.123.100`**? If ping works but there is still no data, the binary is bound to a stale interface name — re-run **`./setup.sh`** and rebuild. This one cause explains nearly every "program runs, nothing happens".
2. **Build fails with missing unitree headers.** Step 2 was skipped or failed — **`unitree_sdk2`** must be installed system-wide before **`setup.sh`** can build.
3. **Arm ignores move commands but telemetry works.** Joints are not enabled — run **`./joint_enable_control`** after every arm power cycle.
4. **send_pose.py rejects an angle.** Working as intended: the pose violates **`limits.py`**. Read Chapter 7 before considering widening a bound.
5. **Arm does something unexpected.** Power off first, ask questions after. Then check the workspace, re-zero, and if anything sounded or looked wrong, record it — that is data (Chapter 7's safety rule).

Summary

One SDK install, one script, one ordered verification: hear the arm, lock it, zero it, then command it through the validated Python path. The machine is now equipped through Phase 7's ROS 2 needs. The next chapter explains what was actually happening under those commands — the network, the middleware, and the protocol.

3

Talking to the Arm

Every capability this project builds — characterization, kinematics, trajectories — ultimately becomes a string of JSON pushed over Ethernet. This chapter explains that pipe from the wire up: the network link, the DDS middleware, and the arm's JSON protocol.

The physical link

The D1 connects to the workstation over Ethernet and lives at a fixed address on a private subnet:

- **Robot**: `192.168.123.100`
- **Host**: `192.168.123.10`, configured as a static profile named `d1-robot` by `setup.sh`

`setup.sh` is the one-command bring-up for a fresh machine. It installs missing packages, verifies the `unitree_sdk2` headers and CycloneDDS are installed, auto-detects which Ethernet interface is the robot link (it works with built-in ports and USB-C adapters), writes the static IP profile, and builds the SDK programs. It is safe to re-run at any time — swapping to a different USB Ethernet adapter and re-running `./setup.sh` is the supported workflow.

> **Important**
>
> Every C++ program binds DDS to the robot's network interface *explicitly*, by name:
> `ChannelFactory::Instance()->Init(0, "enx4cea4168e514");`
> Interface names like `enx4cea4168e514` are derived from the adapter's MAC address, so they change when the adapter does. `setup.sh` patches the current name into the sources automatically; a mysterious "program runs but nothing happens" is almost always a stale interface name.

DDS in one page

The D1 does not expose a REST API or a serial protocol. It speaks **DDS** (Data Distribution Service), the same publish/subscribe middleware that underlies ROS 2 — one reason this project's later ROS 2 phase is a natural fit.

The mental model:

- **Topic**: a named channel, like a radio frequency. Writers publish onto it; readers subscribe to it. Nobody connects to anybody directly.
- **No broker**: there is no central server. Participants discover each other by multicast on the local network segment — which is why the interface binding above matters.
- **Typed messages**: each topic carries one message type. The arm uses two: `ArmString_` (a wrapper around a JSON string) and `PubServoInfo_` (a struct of seven floats).

Three topics matter:

- `rt/arm_Command` — host publishes JSON commands here.

- **`arm_Feedback`** — arm publishes JSON status and acknowledgements here.
- **`current_servo_angle`** — arm publishes raw per-servo angles here at about 10 Hz, as **`PubServoInfo_`** structs.

The concrete DDS implementation is Eclipse CycloneDDS, installed as part of **`unitree_sdk2`**. The Unitree SDK wraps it in two small classes — **`ChannelPublisher`** and **`ChannelSubscriber`** — which are the only DDS API this project touches.

The JSON protocol

Inside every **`ArmString_`** message is a JSON object with the same four-field envelope, fully documented in **`d1_sdk/PROTOCOL.md`**:

```
{"seq": 4, "address": 1, "funcode": 1, "data": { ... }}
```

- **seq**: a tag you choose for commands (the examples all use 4). Feedback the arm volunteers uses **`seq = 10`**; acknowledgements echo the **`seq`** of the command they answer, which is how you match replies to requests.
- **address**: which subsystem — **`1`** = commands to the arm, **`2`** = continuous state feedback from the arm, **`3`** = per-command acknowledgements from the arm.
- **funcode**: the instruction within that subsystem.
- **data**: the payload; its shape depends on the funcode.

Commands (address 1)

- **funcode 1** — single-joint move: **`id`**, **`angle`**, **`delay_ms`** (currently unused, send 0).
- **funcode 2** — all-joint move: **`mode`** plus **`angle0`**…**`angle6`**. Mode 0 means smoothed 10 Hz streaming updates; mode 1 means a trajectory-style move with large smoothing. *This is the workhorse; everything in this project sends funcode 2, mode 1.*
- **funcode 4 / 5** — enable or de-energize one joint / all joints. Nominally **`mode`** 1 = enable, 0 = free, but the official docs reveal it is really a *lock level* from 0 (fully unloaded) to 80000 (fully locked) — a dial, not a switch. De-energized joints plus the angle feedback stream enable **drag teaching**: physically move the arm and record the angles as a demonstration.
- **funcode 6** — motor power switch. Doubles as the software emergency stop.
- **funcode 7** — return to zero posture; takes no **`data`** at all.

A complete all-joint command looks like this:

```
{"seq":4,"address":1,"funcode":2,"data":{"mode":1,
 "angle0":0,"angle1":-30,"angle2":30,"angle3":0,
 "angle4":20,"angle5":0,"angle6":0}}
```

Feedback (address 2) and acknowledgements (address 3)

On **`address 2`** the arm continuously pushes joint angles (funcode 1, at 10 Hz), arm status (funcode 3: enable / power / error flags), and per-motor health (funcode 4). On **`address 3`** it answers each command twice: funcode 1 reports whether the message parsed (**`recv_status`**), funcode 2

whether it executed (**`exec_status`**).

> **Note**
>
> Angles in this protocol are **degrees**, and that convention is kept throughout the repository's tooling. The kinematics math in Phases 4–6 works in radians internally; the conversion happens at the tool boundary, in exactly one place, or bugs multiply.

Recap — the stack

Ethernet carries DDS; DDS carries topics; **`rt/arm_Command`** carries JSON; JSON carries funcodes. When something doesn't move, debug in that order: link up? interface name patched? topic name right? JSON well-formed?

Summary

The arm speaks a small, clean JSON protocol over brokerless publish/subscribe middleware. Two topics in, one topic out, seven angle channels in degrees. The next chapter builds the only two programs that ever touch this layer directly.

4

The C++/Python Bridge

The Unitree SDK is C++ because its DDS bindings are. The project's algorithms are Python because that is where NumPy, analysis, and fast iteration live. This chapter covers the piece that joins them: two tiny C++ programs and a pipe.

The architecture decision

The obvious alternatives were rejected deliberately:

- **Everything in C++**: analysis, plotting, and experiment logic would slow to a crawl. Nobody characterizes a servo faster in C++.
- **Python DDS bindings**: possible, but adds a fragile dependency for zero benefit at this stage.
- **Chosen: thin C++ I/O processes + Unix pipes.** C++ does exactly two jobs — publish commands, subscribe to telemetry — and exposes them as plain text on **`stdin`**/**`stdout`**. Python drives them with **`subprocess`**.

```
Python (algorithms)      C++ bridge (robot I/O)      Robot
-------------------      ----------------------      ------
send_pose.py        -->  joint_commander        -->  DDS
log_joints.py       <--  get_arm_joint_angle    <--  DDS
```

This is a placeholder for the real thing: Phase 7 replaces the pipes with ROS 2 nodes, which is the standard industry pattern. But the shape — C++ owns robot I/O, Python owns math — survives that migration intact.

joint_commander.cpp — the command side

This is the one program written from scratch for Phase 2 (the other SDK examples ship with the arm). Its contract: read lines of seven space-separated angles from **`stdin`** forever; publish each as a funcode 2 command. The full program is about 50 lines.

First, the DDS setup — three lines, boilerplate shared by every SDK example:

`d1_sdk/src/joint_commander.cpp`

```cpp
#define TOPIC "rt/arm_Command"

int main(){
    // Bind DDS to the adapter that faces the robot
    // (setup.sh patches in the machine's real NIC name)
    ChannelFactory::Instance()->Init(0, "enx4cea4168e514");
    ChannelPublisher<unitree_arm::msg::dds_::ArmString_>
        pub(TOPIC);
    pub.InitChannel();
```

Then the loop. Each line is parsed into seven floats; a malformed line prints a warning and is skipped rather than killing the process — the commander must survive sloppy input because interactive use is expected:

```
    std::string line;
    while(std::getline(std::cin, line)){
        if(line.empty()) continue;
        std::istringstream iss(line);
        std::array<float, 7> j;
        bool ok = true;
        for(int i = 0; i < 7; ++i){
            if(!(iss >> j[i])){ ok = false; break; }
        }
        if(!ok){
            std::cerr << "Invalid input. Please enter 7 joint "
                         "angles separated by spaces.\n";
            continue;
        }
```

Finally each valid line becomes a JSON string (funcode 2, mode 1 — the smoothed trajectory-style move) and is published:

```
        std::ostringstream ss;
        ss << "{\"seq\": 4, \"address\": 1, \"funcode\": 2, "
           << "\"data\":{\"mode\":1"
           << ",\"angle0\":" << j[0] << /* ... angle1..angle6 */
           << "}}";

        unitree_arm::msg::dds_::ArmString_ msg{};
        msg.data_() = ss.str();
        pub.Write(msg);
    }
```

That's the whole robot-command surface of the project. Test from a shell:

```
$ echo "0 -30 30 0 20 0 0" | ./joint_commander
```

> **Tip**
> Because the commander exits when **`stdin`** closes, **`echo`**-piping sends exactly one command and cleanly terminates. The arm's own servo controller holds the last commanded position; the commander does not need to stay alive.

get_arm_joint_angle.cpp — the telemetry side

The telemetry program is the mirror image: it subscribes to two topics and prints everything it hears. On **`current_servo_angle`** it receives a **`PubServoInfo_`** struct per message and prints one parseable line:

```
void Handler(const void* msg)
```

```
{
    const unitree_arm::msg::dds_::PubServoInfo_* pm =
        (const unitree_arm::msg::dds_::PubServoInfo_*)msg;

    std::cout << "servo0_data:" << pm->servo0_data_()
              << ", servo1_data:" << pm->servo1_data_()
              /* ... through servo6_data ... */
              << std::endl;
}
```

A second handler prints raw JSON from **`arm_Feedback`** prefixed with **`armFeedback_data:`**. The **`main`** function registers both subscribers and sleeps forever; the SDK dispatches callbacks on background threads.

The key property is that **the output is a line protocol**: one reading per line, fixed field names, flushed by **`std::endl`**. That is what makes the Python side trivial.

Driving the bridge from Python

Python needs only the standard library:

```
import subprocess

# Command side: write lines to joint_commander's stdin
subprocess.run(["d1_sdk/build/joint_commander"],
               input="0 -30 30 0 20 0 0\n", text=True)

# Telemetry side: read lines from get_arm_joint_angle's stdout
proc = subprocess.Popen(["d1_sdk/build/get_arm_joint_angle"],
                        stdout=subprocess.PIPE, text=True)
for line in proc.stdout:
    ...  # parse "servo0_data:12.34, servo1_data:..." lines
```

Everything in Chapter 5 is an elaboration of these two snippets.

Summary

Two small C++ processes translate between DDS and plain text; Python owns everything above them. The design is intentionally disposable — Phase 7 swaps the pipes for ROS 2 topics without changing where the intelligence lives.

5

The Experiment Toolkit

Every experiment in this project reduces to the same three verbs: command a pose, log what actually happened, respect the limits. This chapter walks through the Python tools in **`experiments/`** that implement those verbs, and the design principles that keep them trustworthy.

log_joints.py — the telemetry logger

The logger wraps **`get_arm_joint_angle`** and turns its text stream into timestamped CSVs. Its interface grew with the phases:

```
$ python experiments/log_joints.py                    # until Ctrl-C
$ python experiments/log_joints.py --duration 12      # timed capture
$ python experiments/log_joints.py --label step_j2    # tag the filename
$ python experiments/log_joints.py --outdir logs/phase3
```

Files are named **`joints_<timestamp>_<label>.csv`** so a directory of logs is self-describing. Three implementation details are worth understanding:

Parsing with a regex, defensively

`experiments/log_joints.py`

```
_SERVO_RE = re.compile(r"servo(\d)_data:([-\d.]+)")

def parse_servo_line(line):
    """[s0..s6] from a servo line, or None if not one."""
    matches = _SERVO_RE.findall(line)
    if len(matches) != NUM_SERVOS:
        return None
    pairs = sorted(matches, key=lambda m: int(m[0]))
    return [float(v) for _, v in pairs]
```

The subscriber prints more than servo lines (JSON feedback, for one), so anything that doesn't yield exactly seven **`servoN_data`** fields is silently dropped. Sorting by servo index means the code doesn't depend on field order in the C++ print statement — a cheap insurance policy.

Timestamps and flushing

Each row records **`time.time()`** at parse time — wall-clock timestamps, so logs from different tools can be correlated. The CSV is flushed every 50 rows: recent data survives a crash or Ctrl-C, without paying a disk sync per row at 10 Hz.

Timed capture

`-duration` is implemented with a **`time.monotonic()`** deadline checked per line, and the subprocess is terminated in a **`finally`** block. Monotonic time is the right clock for measuring intervals; wall time can jump (NTP), monotonic cannot.

poses.py and send_pose.py — the command side

`poses.py` is deliberately boring — a dict from name to seven angles, nothing else:

`experiments/poses.py`

```
poses = {
    "zero":         [0, 0, 0, 0, 0, 0, 0],
    # progressive extension; j1 roughly doubles per step
    "reach_1":      [0, -5, 10, 0, 5, 0, 0],
    "reach_2":      [0, -10, 17, 0, 15, 0, 0],  # anchor
    "reach_3":      [0, -20, 25, 0, 18, 0, 0],
    "reach_4":      [0, -30, 30, 0, 20, 0, 0],
    # base yaw both directions, arm in the reach_2 posture
    "yaw_left":     [30, -10, 17, 0, 15, 0, 0],
    "yaw_right":    [-30, -10, 17, 0, 15, 0, 0],
    # wrist rotations (j3, j5) both directions; j6 is the
    # gripper (0-65 mm stroke, never negative): wrist_open
    # opens it to 25, wrist_closed brings it back to 0
    "wrist_open":   [0, -10, 17, 25, 15, -25, 25],
    "wrist_closed": [0, -10, 17, -25, 15, 25, 0],
}
```

Note the design of the later poses: they hold the arm in the proven **`reach_2`** anchor posture and vary only the joints under test. That is the "one variable per experiment" rule showing up in data design.

`send_pose.py` accepts either form and validates before anything touches hardware:

```
$ python experiments/send_pose.py 0 -30 30 0 20 0 0
$ python experiments/send_pose.py reach_2
```

Its core function is the safety choke point of the whole repository:

`experiments/send_pose.py`

```
from limits import LIMITS

def send_pose(angles, commander=COMMANDER):
    """Validate and send 7 joint angles to the arm."""
    if len(angles) != 7:
        raise ValueError(f"Expected 7 angles, got {len(angles)}")

    for i, (a, (lo, hi)) in enumerate(zip(angles, LIMITS)):
        if not lo <= a <= hi:
            raise ValueError(
                f"j{i}={a} deg is outside safe range [{lo}, {hi}]")

    if not commander.exists():
        raise FileNotFoundError(...)
```

```
    cmd = " ".join(f"{a:g}" for a in angles)
    proc = subprocess.run([str(commander)], input=cmd + "\n",
                          text=True, capture_output=True)
```

Order matters here: count check, then limit check, then binary check, then — only then — the subprocess call. Everything that can fail cheaply fails before metal moves.

limits.py — the contract

`experiments/limits.py` holds the confirmed safe envelope as plain constants, and is the file every later phase imports:

`experiments/limits.py`

```
# (min, max) per joint. PROVISIONAL at +/-45 until Phase 3
# sequence runs confirm the envelope. j6 is the gripper:
# 0-65 mm claw stroke per the docs, so it is never negative.
LIMITS = [(-45.0, 45.0)] * 6 + [(0.0, 45.0)]  # condensed

# Measured in Phase 2 (see PHASE2_RESULTS.md):
ZERO_OFFSETS = [0.000, 0.400, 0.393, -0.100,
                0.400, -0.200, -0.200]
NEG_DEADBAND = {2: 0.60, 4: 0.80}  # degrees, j2/j4 negative only
```

The rules for this file: never widen a bound without hardware re-testing; tighten immediately when anything looks wrong. Because **`send_pose()`** is the single path to the robot and it imports **`LIMITS`**, tightening this file instantly constrains every tool in the repository.

> **Important**
>
> This is the "single source of truth" pattern doing safety work. There is exactly one pose dictionary, one limits table, and one function that commands hardware. Earlier drafts had **`send_pose.py`** carrying its own private limits copy — a bug factory eliminated by the import.

Recap — the toolkit's shape

Three files, three responsibilities: **`poses.py`** names positions, **`limits.py`** bounds them, **`send_pose.py`** enforces and sends. **`log_joints.py`** watches everything, independently.

Summary

The toolkit converts raw pipes into safe, labeled, reproducible experiments. With it in place, the project could start asking the arm real questions — which is exactly what Phase 2 did.

6

Phase 2 — What the Joints Actually Do

Phase 2 asks the most basic question in hardware robotics: when you command a joint to an angle, what happens? Not what the datasheet says — what the encoders report. This chapter covers the method, the automation, and the findings, including one genuine discovery.

Why characterize at all

Every later phase silently assumes answers to these questions:

- **Zero offset**: when commanded to 0°, where does the joint actually sit? (FK validation in Phase 4 dies without this.)
- **Settle time**: how long after a command until the reading is stable? (Trajectory timing in Phase 8 needs it.)
- **Overshoot**: does the joint ring past its target? (Safety margins depend on it.)
- **Steady-state error**: where does it stop relative to the command? (IK tolerance budgets start here.)
- **Coupling**: does moving one joint disturb the others? (Multi-joint control assumes no.)

"Commanded ≠ measured" is the theme. The point of Phase 2 is to replace assumptions with a table of numbers.

The step-response method

The instrument is a **step input** — the classic tool of control engineering. Hold everything at zero, command one joint to jump to a target, and record the full time history of its response. From that curve you read off settle time (when it enters and stays within a band around the target, here ±0.5°), overshoot (does it cross the target and come back), and steady-state error (where it finally rests).

Per joint, the procedure was: log a 5 s baseline at zero; step to +10°, log 12 s; back to zero; step to −10°; step to +30°; then log 5 s at zero again to confirm clean return. Seven joints, three steps each, plus baselines — 21 step tests and 14 static logs, all automated by **`run_phase2.sh`** in about eight minutes.

The logger-first pattern

The orchestration detail that makes the data good:

`experiments/run_phase2.sh (core pattern)`

```
python3 log_joints.py --duration 12 --label step_j2_pos10 &
sleep 2                        # capture the "before" state
echo "0 0 10 0 0 0 0" | ./joint_commander
wait                           # logger finishes its 12 s
```

The logger starts *two seconds before* the command. Every CSV therefore contains the pre-step baseline, the transient, and the settled tail — the complete story, with no guessing about when the step happened. This pattern recurs in Phase 3's sequence runner.

The results

Full numbers live in **`experiments/PHASE2_RESULTS.md`**; the analysis was produced by **`plot_phase2.py`** from 45 CSVs in **`logs/phase2/`**.

Zero offsets

Commanded to zero, joints read between $-0.2°$ and $+0.4°$ (j0 is a perfect 0.000; j1, j2, j4 sit around $+0.4°$). Small, but systematic — so they are recorded in **`limits.py`** as **`ZERO_OFFSETS`** and will be subtracted by any controller that needs precision.

Step response character

- **Settle time**: $\pm 10°$ steps settle in 0.45–0.67 s; $\pm 30°$ steps in 1.78–1.89 s.
- **All responses are overdamped** — no joint overshoots, ever. Practically, this means moves can be commanded back-to-back without waiting out oscillations, and a $60°$ swing can be bounded by extrapolating settle time rather than fearing ring-out.
- **Steady-state error**: between $0.0°$ and $0.8°$ depending on joint and direction. j0 is the most accurate joint; j4 in the negative direction is the worst.

> **Note**
>
> **Overdamped**, in control terms: the system approaches its target without crossing it, like a door with a strong closer. The opposite — underdamped — overshoots and oscillates. Overdamped is the friendly case for safety, at the cost of some speed.

The deadband discovery

The interesting finding: **j2 and j4 cannot reach small negative targets**. Commanded to $-10°$, j2 lands at $-9.40°$ and j4 at $-9.20°$ — every single time.

The "every single time" is the discovery. A five-trial repeatability test (**`test_deadband.sh`**) measured the spread across trials: **0.00**°. That number does the diagnostic work:

- **Mechanical friction** is messy — it would scatter the landing point across trials.
- **A controller deadband** is exact — firmware stops commanding torque once position error drops below a threshold, so the joint parks at precisely the same shortfall every time.

Zero spread means deadband, not damage: the arm's internal controller gives up $0.60°$ early on j2 and $0.80°$ early on j4, in the negative direction only. No hardware problem, and the fix is pure software — to land j2 at exactly $-10.0°$, command $-10.6°$. These corrections are **`NEG_DEADBAND`** in **`limits.py`**.

> **Note**
>
> "Firmware" here is more approachable than the word suggests: the D1's controller is a small Linux computer running Unitree's **`marm_*`** services, and per the official docs its

driver source (**`marm_code/src/`**) is reachable over SSH. A future side-quest could locate the position-error threshold in **`marm_control`** and confirm this deadband at its source. Until then, the measured correction table is the contract.

Coupling

None detected. With any single joint commanded, all others stayed inside their zero-offset noise floor ($< 0.1°$). In the tested range the joints are mechanically independent — which licenses the multi-joint work of Phase 3.

Recap — what Phase 2 bought

A correction table (offsets + deadband), trust in the servo controller (overdamped, repeatable), timing constants for everything that follows, and a demonstrated method: measure, hypothesize, re-measure to discriminate.

Summary

Seven joints characterized by automated step-response testing. The arm is accurate to better than a degree, never overshoots, and hides one firmware quirk — a negative-direction deadband on j2 and j4 — now documented and correctable. Phase 3 builds on exactly these numbers.

7

Phase 3 — The Safe Envelope

Phase 2 moved one joint at a time. Phase 3 moves all of them, and its product is a contract: **`limits.py`**, the per-joint bounds inside which every future phase — IK included — is allowed to operate. This chapter explains the pose set, the sequence experiment, and the path-dependence analysis that decides whether the envelope can be trusted.

The question Phase 3 answers

Before an IK solver is allowed to pick arbitrary joint combinations, two things must be established empirically:

1. **Reachability without harm**: a set of multi-joint poses, spanning every joint in both directions, that the arm reaches without strain, odd sounds, or collision.
2. **Path independence**: the arm lands in the *same place* for a given pose regardless of where it came from. If arriving at **`reach_3`** from **`zero`** gives a different settled position than arriving from **`reach_4`**, then position commands don't mean what they appear to mean, and everything downstream inherits the ambiguity.

Path dependence in a real arm comes from physical effects: gear backlash (slack that loads differently depending on approach direction), stiction, and gravity sag that varies with configuration. The Phase 2 deadband is a small example already in hand — approach direction changing the landed position.

The pose set and its coverage

The nine poses in **`poses.py`** (shown in Chapter 5) were designed around a coverage argument, all inside the $\pm 45°$ conservative ceiling:

- **zero, reach_1 … reach_4**: a progression of increasingly extended reach postures, j1 roughly doubling per step (-5, -10, -20, $-30°$) while j2 rises to $+30°$ and j4 to $+20°$. **`reach_2`** is the known-good anchor the remaining poses hold.
- **yaw_left / yaw_right**: base joint j0 at $\pm 30°$, with the arm held in the **`reach_2`** anchor posture.
- **wrist_open / wrist_closed**: wrist rotations j3 and j5 at $\pm 25°$ with mirrored signs, same posture. The gripper j6 is exercised alongside them — but only on its documented side: **`wrist_open`** opens it to 25, **`wrist_closed`** brings it back to 0. An earlier draft of the pose set commanded j6 to -25, which is outside the gripper's 0–65 mm stroke entirely; Phase 2 showed the controller tolerates negative j6, but a tolerated out-of-range command is not a safe envelope, so **`limits.py`** now pins j6 to $[0, 45]$.

Known gaps, on purpose: j1 positive and j2/j4 negative are unexercised, because the original pose set only moved those joints one way and the untested directions may drive the arm toward the table or itself. The envelope those directions get is whatever Phase 2's single-joint $-10°$ tests justify, until poses are added deliberately.

run_sequence.py — the experiment driver

The Phase 3 procedure — send each pose, log 10 s, visually confirm, repeat in multiple orders — is automated by **`experiments/run_sequence.py`**. The orderings are *derived* from the pose dict, so new poses join the experiment automatically:

`experiments/run_sequence.py`

```
_ALL = list(poses)
ORDERS = {
    "A": _ALL,                    # as defined (starts at zero)
    "B": _ALL[::-1],              # reversed (ends at zero)
    "C": _ALL[::2] + _ALL[1::2],  # interleaved: bigger jumps
}
```

Within a sequence, poses run **back-to-back** — deliberately not re-zeroing between steps, because the pose-to-pose transitions are exactly what path-dependence testing needs. The arm re-zeros and settles only between sequences, so each ordering starts from the same state.

Each step reuses the logger-first pattern from Phase 2 and then puts a human in the loop:

```
proc, csv_path = start_logger(label)   # 2 s baseline first
time.sleep(PRE_WAIT)
send_pose(angles)                      # validated, then sent
proc.communicate()                     # 10 s observation
note = input("Strain / odd sound / collision? "
             "Enter = clean, or describe: ").strip()
```

The observation prompt is not decoration — it implements the Phase 3 safety rule. A typed note is recorded in the run's markdown log and offers an abort.

> **Important**
>
> On abort or Ctrl-C the arm is deliberately **left where it is**. An automatic return-to-zero sounds helpful, but if the arm has just collided with something, driving it back through the obstacle makes things worse. The operator inspects, then re-zeros manually with **`python experiments/send_pose.py zero`**.

Every run — complete or aborted — writes **`logs/phase3/sequence_log_<stamp>.md`**: one row per step with pose, angles, CSV filename, and the operator's observation. That file is the phase's required "sequence log" artifact.

analyze_sequences.py — the verdict

After the runs, **`analyze_sequences.py`** reads every step CSV, takes the **mean of the final two seconds** as the settled position (safe because Phase 2 showed worst-case settling under 2 s, leaving ≥ 8 s of settled data in each 10 s observation), and groups by pose: where did **`reach_3`** land when reached via order A, B, and C?

The decision rule: per joint, the **spread** (max minus min across orderings) must stay within 1.0°. The threshold comes from Phase 2, not from taste — steady-state errors up to 0.8° and deadbands of 0.6–0.8° are already known repeatability, so only spread beyond that indicates a path effect. Anything flagged gets re-run before **`limits.py`** is touched, to rule out one-off noise.

Tip

The analyzer was validated before ever seeing real data, by feeding it synthetic CSVs with a known 2° path-dependence injected on one joint of one pose. It flagged exactly that joint and nothing else. Test your instruments before trusting what they say about the world.

Exit criteria

Phase 3 closes when: the poses are defined and each has been executed and visually confirmed clean; the three orderings have run; the analyzer reports all spreads $\leq 1.0°$ (or the exceptions are understood and documented); and **`limits.py`** is committed with its PROVISIONAL note replaced by confirmed bounds. Then the tag **`phase3-envelope-frozen`** is laid down and the envelope stops being negotiable.

Summary

Phase 3 turns "the arm can reach these poses" into a tested, frozen contract. Nine poses cover every joint, three orderings probe path dependence, a human confirms each step, and a validated analyzer makes the call against a threshold derived from Phase 2 data. What comes out is the single file every future phase imports before moving metal.

8

The Road Ahead

Everything so far has been careful plumbing and measurement. The phases ahead are where the robotics theory arrives. This chapter previews each one — what it builds, the core idea in plain terms, and what to read — so the mathematics never lands cold.

What to be comfortable with first

The common foundation for Phases 4–6 is linear algebra, used concretely:

- **Matrix multiplication as composition**: chaining transforms means multiplying matrices, in order. Order matters.
- **Rotation matrices** (3×3, orthonormal) and **homogeneous transforms** (4×4, rotation plus translation — the set called SE(3)).
- **Cross products** and their matrix form (the "skew" matrix), which is how rotation axes enter the algebra.
- **Radians** internally, always; degrees only at the tool boundary.
- **NumPy fluency**: `@`, broadcasting, **`np.linalg`**.

The theory source for the whole arc is *Modern Robotics* by Lynch & Park (free PDF online). Each phase below names its chapter.

Phase 4 — forward kinematics

The question: given seven joint angles, where is the end-effector in 3D space?

The implementation uses the **Product of Exponentials** (POE) formula rather than the older Denavit–Hartenberg tables. The intuition: each joint is a hinge in space, described by a **screw axis** — a 6-vector $S = [\omega, v]$ bundling the rotation axis and where it sits. Rotating joint i by θ_i transforms everything beyond it by the matrix exponential $e^{[S_i]\theta_i}$. Chain all seven, close with M (the end-effector pose at zero configuration), and that is FK:

```
T(theta) = exp([S1] th1) exp([S2] th2) ... exp([Sn] thn) M
```

The screw axes and M come from the D1's URDF file (already in the repository), parsed with Python's XML tools. Validation is physical: tape a grid to the table, command three known configurations, and require the prediction to land within 2 cm of where the arm's tip actually is — which is why Phase 2's zero offsets and Phase 3's safe poses had to come first. *Read: Lynch & Park Chapter 4 (and Chapter 3 for Rodrigues' rotation formula).*

Phase 5 — Jacobians and differential IK

The question: to nudge the end-effector 1 cm in $+x$, how should the joints move?

The **Jacobian** $J(\theta)$ is the matrix that maps joint velocities to end-effector velocity at the current configuration — column i is joint i's screw axis re-expressed at the current pose. Differential IK inverts the map. Because J can be non-square and near-singular, the inversion uses **damped least squares**, $J^T(JJ^T + \lambda^2 I)^{-1}$, which trades a little accuracy for not exploding near singular poses (configurations where some direction of motion is momentarily impossible, like a fully stretched arm being asked to extend further). Hardware target: a commanded 1 cm Cartesian step lands within 5 mm. *Read: Lynch & Park Chapter 5.*

Phase 6 — numerical IK

The question: given a target pose, what joint angles reach it?

There is no closed-form answer for a general arm; instead, iterate differential IK Newton-style — compute the pose error as a twist (via the matrix log, the inverse of Phase 4's exponential), take a damped step, clamp to **`LIMITS`**, repeat until converged. The engineering is in the failure handling: convergence tolerances split between rotation and translation, retry seeds when iteration stalls, and an automated suite of 200 random reachable targets with a $\geq$ 90% success requirement. **`limits.py`** appears here in its final role: the clamp inside every IK step. *Read: Lynch & Park Chapter 6.*

Phase 7 — ROS 2

ROS 2 replaces the stdin/stdout bridge with the industry-standard middleware — which is DDS underneath, the same technology the D1 already speaks. The vocabulary: a **node** is a process that does one thing; **topics** are pub/sub streams (a **`joint_state_publisher`** node will publish the arm's angles on **`/joint_states`**); **services** are request/reply (FK becomes a service); **actions** are long-running goals with feedback (trajectories, later). The first milestone is RViz showing the URDF model tracking the real arm live. The repository's **`ROS2/cheatsheet.md`** holds the working notes.

ROS 2 distributions pair with Ubuntu LTS releases; on this project's Ubuntu 26.04 the matching distribution is **ROS 2 Lyrical Luth**. Install the official binary packages from docs.ros.org — never build ROS from source for this — plus **`python3-colcon-common-extensions`**. If the workstation OS ever changes, the distribution changes with it; everything conceptual in this section survives that swap untouched. *Read: the official docs.ros.org beginner tutorials, all of them — one day that saves a week.*

Phases 8–11 — trajectories, simulation, perception, capstone

Trajectories (Phase 8): replace "go to pose" with a timed path. Cubic time scaling $s(t) = 3s^2 - 2s^3$ (zero velocity at endpoints) or quintic (zero acceleration too) shapes the motion; an executor streams waypoints through the commander. Gravity compensation is empirical: hold poses, measure droop versus angle, fit a $k \sin\theta$ correction — the same measure-then-model pattern as Phase 2.

Simulation (Phase 9): load the URDF into MuJoCo, require FK agreement within 5 mm, then run identical trajectories in sim and on hardware and document the gap.

Perception (Phase 10): calibrate a webcam (reprojection error < 1 px), detect ArUco markers for 6-DOF object pose, transform camera coordinates into the robot base frame, publish on a ROS 2 topic.

Capstone (Phase 11): chain everything — detect, IK, plan, execute, place — and measure it honestly: 20 trials, success rate and grasp-position error with mean and standard deviation, target $\geq 15/20$.

Summary

The arc runs measurement → model → control → integration: characterize the joints, learn where the end-effector is (FK), learn how to move it (Jacobians, IK), industrialize the plumbing (ROS 2), make motion smooth (trajectories), and close the loop with eyes (perception). Every stage stands on the measured foundations of Phases 2 and 3 — which is the whole argument for having built them carefully.

Appendix A

Quick Reference

Everything in this appendix is the perishable layer — exact names, versions, and numbers as of this edition's repository revision (see the colophon). When something here disagrees with the repository, trust the repository and update this appendix.

Environment

- **OS**: Ubuntu 26.04 LTS (Resolute Raccoon)
- **ROS 2**: Lyrical Luth (Phase 7 onward; binary install)
- **DDS**: Eclipse CycloneDDS via **`unitree_sdk2`**
- **Python**: system Python 3; NumPy/matplotlib/pandas for analysis
- **Network**: robot **`192.168.123.100`**, host **`192.168.123.10`** (profile **`d1-robot`**)

Bring-up and smoke test

```
$ ./setup.sh                      # fresh machine or new adapter
# power the arm, wait 60-90 s, then from d1_sdk/build/:
$ ./get_arm_joint_angle           # verify telemetry FIRST
$ ./joint_enable_control          # lock joints
$ ./arm_zero_control              # move to zero posture
```

Everyday commands

```
# send a named pose or raw angles (validated against limits.py)
$ python experiments/send_pose.py zero
$ python experiments/send_pose.py 0 -30 30 0 20 0 0

# log telemetry to CSV
$ python experiments/log_joints.py --duration 12 --label mytest

# raw one-liner, bypassing Python (no limit checks -- careful)
$ echo "0 0 0 0 0 0 0" | d1_sdk/build/joint_commander
```

Phase experiment workflows

```
# Phase 2: full characterization (~8 min) and analysis
$ bash experiments/run_phase2.sh
$ python3 experiments/plot_phase2.py --summary --table
$ bash experiments/test_deadband.sh        # j2/j4 recheck

# Phase 3: sequence runs and path-dependence verdict
$ python experiments/run_sequence.py        # or --only B
$ python experiments/analyze_sequences.py
```

File map

- **`experiments/poses.py`** — named poses (degrees on j0–j5; j6 is gripper stroke, ≥ 0).
- **`experiments/limits.py`** — **`LIMITS`**, **`ZERO_OFFSETS`**, **`NEG_DEADBAND`**. The contract.
- **`experiments/send_pose.py`** — the only path to the robot; validates first.
- **`experiments/log_joints.py`** — telemetry to CSV; **`-duration`**, **`-label`**, **`-outdir`**.
- **`experiments/run_sequence.py`** / **`analyze_sequences.py`** — Phase 3 driver and verdict.
- **`experiments/PHASE2_RESULTS.md`** — characterization numbers.
- **`d1_sdk/PROTOCOL.md`** — full JSON protocol reference.
- **`logs/phaseN/`** — raw CSVs, immutable.

Protocol crib sheet

Envelope: **`{"seq": n, "address": a, "funcode": f, "data": {...}}`**. Addresses: 1 = command (host $\rightarrow$ arm), 2 = state feedback (arm $\rightarrow$ host, **`seq`**=10), 3 = command ack. Commands on address 1: funcode 1 single joint, **2 all joints (the one used here)**, 4/5 enable one/all, 6 power, 7 zero posture. Feedback on address 2: funcode 1 angles at 10 Hz, 3 status flags, 4 motor health. Acks on address 3: funcode 1 received-OK, 2 executed-OK.

Measured constants (Phase 2)

- Settle: ≤ 0.7 s for 10° steps, ≤ 1.9 s for 30° steps; all joints overdamped, zero overshoot.
- Zero offsets: j1/j2/j4 $\approx +0.4^\circ$; j5/j6 $\approx -0.2^\circ$; j0 exact.
- Deadband: j2 stops 0.60° short and j4 0.80° short on negative targets — perfectly repeatable, firmware, not damage.
- Coupling: none above the 0.1° noise floor.

Factory constants (D1-550 official spec)

- Range of motion: j0/j3/j5 $\pm 135^\circ$; j1/j2/j4 $\pm 90^\circ$; j6 gripper 0–65 mm stroke. *Never approached in this project* — the working envelope is **`limits.py`**.
- Torque: 3.3 Nm (j0, j1); 1.7 Nm (j2–j6). Payload 500 g including gripper. Reach 550 mm.
- Control cycle and telemetry rate: 10 Hz. Power: 24 V, 240 W.
- Arm IP **`192.168.123.100`**; onboard Linux, SSH user **`ubuntu`**.

References

- Official D1 services interface (protocol, specs, D1-T setup, multi-arm control): https://support.unitree.com/home/en/developer/D1Arm_services
- D1 SDK + sample programs (requires **`unitree_sdk2`**): **`d1_sdk.zip`** from Unitree's firmware CDN, linked from the page above. URDF and meshes: the "Robotic Arm Download" links on the same page (gripper version already vendored in this repository).
- *Modern Robotics*, Lynch & Park — free PDF online; chapter map in Chapter 8.
- ROS 2 Lyrical Luth: https://docs.ros.org